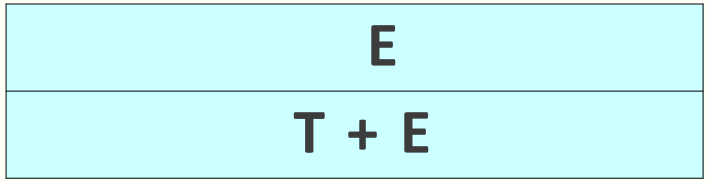
《编译原理》

第12讲b：语法分析–LL(k)

何冬杰
重庆大学

Predictive Parsing



$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow int$

$T \rightarrow (E)$



Predictive Parsing

$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow int$

$T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)

int	+	(	int	+	int	)

Predictive Parsing

$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow int$

$T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)



int	+	(	int	+	int	)	

Predictive Parsing

- Leftmost DFS and BFS use ***backtracking***.
  - Try a production. If it fails, backtrack and try another.
- Predictive parsers select a production ***without backtracking***.
  - Use **lookahead** from the remaining input to make that choice.
- Tradeoffs in Prediction
  - Predictive parsers are ***fast***.
    - Many predictive algorithms can be made to run in linear time.
    - Often can be table-driven for extra performance.
  - Predictive parsers are ***weak***.
    - Not all grammars can be accepted by predictive parsers.
  - Trade expressiveness for speed.

Implementing Predictive Parsing

- Given just the start symbol, how do you know which productions to use to get to the input program?
  - **Idea**: Use lookahead tokens.
  - When trying to decide which production to use, look at some number of tokens of the input to help make the decision.
- Predictive parsing is **only possible** if we can predict which production to use given some number of lookahead tokens.
  - **Increasing** the number of lookahead tokens *increases* the number of grammars we can parse, but *complicates* the parser.
  - **Decreasing** the number of lookahead tokens *decreases* the number of grammars we can parse, but *simplifies* the parser.

A Simple Predictive Parser: LL(1)

- Top-down, predictive parsing:
  - **L**: Left-to-right scan of the tokens
  - **L**: Leftmost derivation.
  - **(1):** One token of lookahead
- Construct a leftmost derivation for the sequence of tokens.
- When expanding a nonterminal, we predict the production to use by looking at the next token of the input.
  - Need a **parsing table**.
  - **The decision is forced**.

LL(1) Parsing Table

- LL(1) predictive parsing table:
  - With **rows** of nonterminals, **columns** of terminals
  - **Table entries** are productions
  - If each cell contains at most one production, parsing is predictive! Table tells us exactly which production to apply.
  - When parsing nonterminal X, and next token is t, entry for X and t will tell us which production to use
- Example:

E → int

E → (E Op E)

Op → +

Op → *

	int	(	)	+	*
E	int	(E Op E)			
Op				+	*

LL(1) Parsing

E$	(int + (int * int))$

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

The $ symbol is an end-of-input marker used by the parser to detect the end of the input and is ***not part of*** the grammar.

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

When the first symbol of our guess is a ***nonterminal***, we consult the parsing table to determine which production to use.

This is called a **predict** step.

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$

When the first symbol of our guess is a ***terminal***, we match it against the first symbol of the remaining input string.

This is called a **match step**.

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$
int Op E)$	int + (int * int))$
Op E)$	+ (int * int))$

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$
int Op E)$	int + (int * int))$
Op E)$	+ (int * int))$
+ E)$	+ (int * int))$
E)$	(int * int))$

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$
int Op E)$	int + (int * int))$
Op E)$	+ (int * int))$
+ E)$	+ (int * int))$
E)$	(int * int))$
(E Op E))$	(int * int))$
E Op E))$	int * int))$

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$
int Op E)$	int + (int * int))$
Op E)$	+ (int * int))$
+ E)$	+ (int * int))$
E)$	(int * int))$
(E Op E))$	(int * int))$
E Op E))$	int * int))$
int Op E))$	int * int))$
Op E))$	* int))$

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int * int))$
(E Op E)$	(int + (int * int))$
E Op E)$	int + (int * int))$
int Op E)$	int + (int * int))$
Op E)$	+ (int * int))$
+ E)$	+ (int * int))$
E)$	(int * int))$
(E Op E))$	(int * int))$
E Op E))$	int * int))$
int Op E))$	int * int))$
Op E))$	* int))$
* E))$	* int))$
E))$	int))$

LL(1) Parsing

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

E$	(int + (int
(E Op E)$	(int + (int
E Op E)$	int + (int
int Op E)$	int + (int
Op E)$	+ (int
+ E)$	+ (int * int))$
E)$	(int * int))$
(E Op E))$	(int * int))$
E Op E))$	int * int))$
int Op E))$	int * int))$
Op E))$	* int))$
* E))$	* int))$
E))$	int))$
int))$	int))$
$	$



LL(1) Error Detection

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$		int + int$
int $		int + int$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

LL(1) Error Detection

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$	int + int$
int $	int + int$
$	+ int$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table



LL(1) Error Detection, Part II

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$	(int (int))$
(E Op E)$	(int (int))$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

LL(1) Error Detection, Part II

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$	(int (int))$
(E Op E)$	(int (int))$
E Op E)$	int (int))$
int Op E)$	int (int))$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table

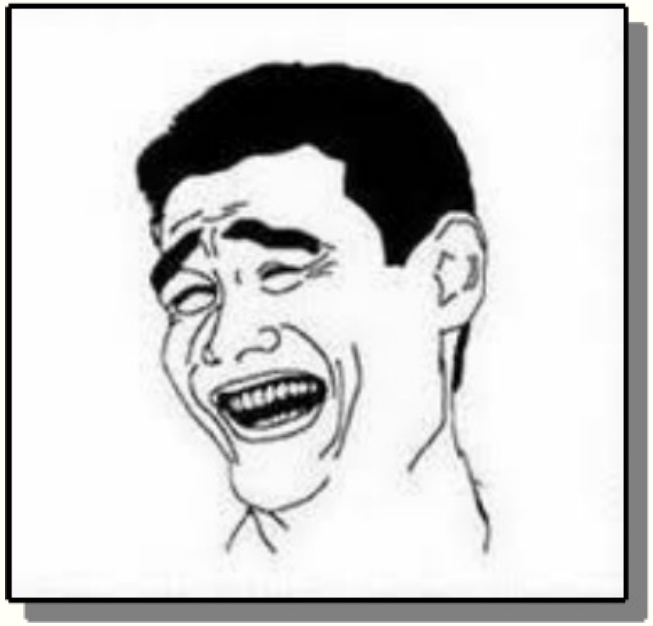
LL(1) Error Detection, Part II

(1) E → int
(2) E → (E Op E)
(3) Op → +
(4) Op → *

E$	(int (int))$
(E Op E)$	(int (int))$
E Op E)$	int (int))$
int Op E)$	int (int))$
Op E)$	(int))$

	int	(	)	+	*
E	1	2			
Op				3	4

Parsing Table



The LL(1) Algorithm

- Suppose a grammar has a start symbol **S** and LL(1) parsing table **T**.
- We want to parse the string **ω**.
- Initialize a stack containing **S$**.
- Repeat until the stack is empty:
  - Let the next character of **ω** be **t**.
  - If the top of the stack is a terminal **r**:
    - If **r** and **t** don't match, report an error.
    - Otherwise, consume the character **t** and pop **r** from the stack.
  - Otherwise, the top of the stack is a nonterminal **A**:
    - If **T[A, t]** is undefined, report an error.
    - Replace the top of the stack with **T[A, t]**.

A Simple LL(1) Grammar

```
STMT  →  if EXPR then STMT
      |  while EXPR do STMT
      |  EXPR ;

EXPR  →  TERM -> id
      |  zero? TERM
      |  not EXPR
      |  ++ id
      |  -- id

TERM  →  id
      |  constant
```

```
id -> id;

while not zero? id
    do --id;

if not zero? id then
    if not zero? id then
        constant -> id;
```

Constructing LL(1) Parse Tables

```
STMT  →  if EXPR then STMT          (1)
      |  while EXPR do STMT         (2)
      |  EXPR ;                     (3)

EXPR  →  TERM -> id                 (4)
      |  zero? TERM                 (5)
      |  not EXPR                   (6)
      |  ++ id                      (7)
      |  -- id                      (8)

TERM  →  id                         (9)
      |  constant                   (10)
```

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR												
TERM												

Constructing LL(1) Parse Tables

```
STMT  →  if EXPR then STMT          (1)
      |  while EXPR do STMT         (2)
      |  EXPR ;                     (3)

EXPR  →  TERM -> id                 (4)
      |  zero? TERM                 (5)
      |  not EXPR                   (6)
      |  ++ id                      (7)
      |  -- id                      (8)

TERM  →  id                         (9)
      |  constant                   (10)
```

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)									
EXPR					(5)	(6)	(7)	(8)				
TERM										(9)	(10)	

Constructing LL(1) Parse Tables

```
STMT  →  if EXPR then STMT          (1)
      |  while EXPR do STMT         (2)
      |  EXPR ;                     (3)

EXPR  →  TERM -> id                 (4)
      |  zero? TERM                 (5)
      |  not EXPR                   (6)
      |  ++ id                      (7)
      |  -- id                      (8)

TERM  →  id                         (9)
      |  constant                   (10)
```

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)									
EXPR					(5)	(6)	(7)	(8)		(4)	(4)	
TERM										(9)	(10)	

Constructing LL(1) Parse Tables

```
STMT  →  if EXPR then STMT          (1)
      |  while EXPR do STMT         (2)
      |  EXPR ;                     (3)

EXPR  →  TERM -> id                 (4)
      |  zero? TERM                 (5)
      |  not EXPR                   (6)
      |  ++ id                      (7)
      |  -- id                      (8)

TERM  →  id                         (9)
      |  constant                   (10)
```

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)	(2)			(3)	(3)	(3)	(3)		(3)	(3)	
EXPR					(5)	(6)	(7)	(8)		(4)	(4)	
TERM										(9)	(10)	

How Do We Construct LL(1) Parse Tables?

- **Intuition**: The next character should uniquely identify a production, so we should pick a production that ultimately starts with that character.

- T[**A**, **t**] should be a production **A** → ω iff ω derives something starting with **t**.
  - More rigorously, T[**A**, **t**] =ω iff **A** → ω and ω → * **t**ω'
  - Assume No **ε**-Productions for Now

- We therefore want to tell if a particular nonterminal **A** derives a string starting with a terminal **t**.
  - We can formalize this with **FIRST sets**.

FIRST Sets

- **FIRST(A)** is the set of terminals that can be at the start of a string produced by **A**

$$\mathbf{FIRST}(\mathbf{A}) = \{\mathbf{t} \mid \mathbf{A} \Longrightarrow^* \mathbf{t}\boldsymbol{\omega} \text{ for some } \boldsymbol{\omega}\}$$

  - If we can compute **FIRST** sets for all nonterminals in a grammar, we can efficiently construct the LL(1) parsing table.

- Computing **FIRST** Sets via a fixed-point iteration
  - Initially, for all non-terminals **A**, $\mathrm{FIRST}(\mathbf{A}) = \{\mathbf{t} \mid \mathbf{A} \rightarrow \mathbf{t}\boldsymbol{\omega} \text{ for some } \boldsymbol{\omega}\}$
  - Then, repeat the following until no changes occur: for each nonterminal **A**, for each production $\mathbf{A} \rightarrow \mathbf{B}\boldsymbol{\omega}$, set $\mathbf{FIRST}(\mathbf{A}) = \mathbf{FIRST}(\mathbf{A}) \cup \mathbf{FIRST}(\mathbf{B})$

Iterative FIRST Computations

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | constant
```

STMT	EXPR	TERM

Iterative FIRST Computations

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | constant
```

STMT	EXPR	TERM
if while	zero? not ++ --	id constant

Initially, $\mathbf{FIRST}(\mathbf{A}) = \{\mathbf{t} \mid \mathbf{A} \Longrightarrow^* \mathbf{t}\boldsymbol{\omega} \text{ for some } \boldsymbol{\omega}\}$

Iterative FIRST Computations

```
STMT →   if EXPR then STMT
       |  while EXPR do STMT
       |  EXPR ;

EXPR →   TERM -> id
       |  zero? TERM
       |  not EXPR
       |  ++ id
       |  -- id

TERM →   id
       |  constant
```

STMT	EXPR	TERM
if while zero? not ++ --	zero? not ++ --	id constant

Iteratively, for $\mathbf{A} \rightarrow \mathbf{B}\boldsymbol{\omega}$, set $\mathbf{FIRST(A)} = \mathbf{FIRST(A)} \cup \mathbf{FIRST(B)}$

Iterative FIRST Computations

```
STMT →   if EXPR then STMT
       |  while EXPR do STMT
       |  EXPR ;

EXPR →   TERM -> id
       |  zero? TERM
       |  not EXPR
       |  ++ id
       |  -- id

TERM →   id
       |  constant
```

STMT	EXPR	TERM
if while zero? not ++ --	zero? not ++ -- id constant	id constant

Iteratively, for $\mathbf{A} \rightarrow \mathbf{B}\boldsymbol{\omega}$, set $\mathbf{FIRST}(\mathbf{A}) = \mathbf{FIRST}(\mathbf{A}) \cup \mathbf{FIRST}(\mathbf{B})$

Iterative FIRST Computations

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | constant
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

Iteratively, for $\mathbf{A} \rightarrow \mathbf{B}\boldsymbol{\omega}$, set $\mathbf{FIRST(A)} = \mathbf{FIRST(A)} \cup \mathbf{FIRST(B)}$

Iterative FIRST Computations

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | constant
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT      (1)
      |  while EXPR do STMT     (2)
      |  EXPR ;                 (3)

EXPR  →  TERM -> id             (4)
      |  zero? TERM             (5)
      |  not EXPR               (6)
      |  ++ id                  (7)
      |  -- id                  (8)

TERM  →  id                     (9)
      |  constant               (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR												
TERM												

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT        (1)
      |  while EXPR do STMT       (2)
      |  EXPR ;                   (3)

EXPR  →  TERM -> id               (4)
      |  zero? TERM               (5)
      |  not EXPR                 (6)
      |  ++ id                    (7)
      |  -- id                    (8)

TERM  →  id                       (9)
      |  constant                 (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)									
EXPR												
TERM												

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT        (1)
      |  while EXPR do STMT       (2)
      |  EXPR ;                   (3)

EXPR  →  TERM -> id               (4)
      |  zero? TERM               (5)
      |  not EXPR                 (6)
      |  ++ id                    (7)
      |  -- id                    (8)

TERM  →  id                       (9)
      |  constant                 (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)		(3)	(3)	(3)	(3)		(3)	(3)	
EXPR												
TERM												

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT      (1)
      |  while EXPR do STMT     (2)
      |  EXPR ;                 (3)

EXPR  →  TERM -> id             (4)
      |  zero? TERM             (5)
      |  not EXPR               (6)
      |  ++ id                  (7)
      |  -- id                  (8)

TERM  →  id                     (9)
      |  constant               (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)		(3)	(3)	(3)	(3)		(3)	(3)	
EXPR										(4)	(4)	
TERM												

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT        (1)
      |  while EXPR do STMT       (2)
      |  EXPR ;                   (3)

EXPR  →  TERM -> id               (4)
      |  zero? TERM               (5)
      |  not EXPR                 (6)
      |  ++ id                    (7)
      |  -- id                    (8)

TERM  →  id                       (9)
      |  constant                 (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)		(3)	(3)	(3)	(3)		(3)	(3)	
EXPR					(5)	(6)	(7)	(8)		(4)	(4)	
TERM												

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT      (1)
      |  while EXPR do STMT     (2)
      |  EXPR ;                 (3)

EXPR  →  TERM -> id             (4)
      |  zero? TERM             (5)
      |  not EXPR               (6)
      |  ++ id                  (7)
      |  -- id                  (8)

TERM  →  id                     (9)
      |  constant               (10)
```

STMT	EXPR	TERM
if while zero? not ++ -- id constant	zero? not ++ -- id constant	id constant

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)		(3)	(3)	(3)	(3)		(3)	(3)	
EXPR					(5)	(6)	(7)	(8)		(4)	(4)	
TERM										(9)	(10)	

From FIRST Sets to LL(1) Tables

```
STMT  →  if EXPR then STMT        (1)
      |  while EXPR do STMT       (2)
      |  EXPR ;                   (3)

EXPR  →  TERM -> id
      |  zero? TERM
      |  not EXPR
      |  ++ id
      |  -- id

TERM  →  id                       (9)
      |  constant                 (10)
```

STMT	EXPR	TERM
if	zero? not ++ -- id constant	id constant



	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	(1)		(2)		(3)	(3)	(3)	(3)		(3)	(3)	
EXPR					(5)	(6)	(7)	(8)		(4)	(4)	
TERM										(9)	(10)	

ε-Free LL(1) Parse Tables

- Given string γ of terminal and non-terminal symbols, **FIRST**(γ) is
  - the set of all *terminal symbols* that can start a string derived from γ
- We can use **FIRST** sets to determine which production to use!

> Given a nonterminal X, and all its productions $X \to \gamma_1, X \to \gamma_2, \ldots, X \to \gamma_n$ with $\gamma_i \neq \varepsilon$, if FIRST($\gamma_1$), …, FIRST($\gamma_n$) all ***mutually disjoint***, then next character tells us which production to use.
>
> ***Is this still correct?***

- **Algorithm** for constructing an LL(1) parse table for a grammar
  - Compute the **FIRST** sets for all nonterminals in the grammar
  - For each production $\mathbf{A} \to \mathbf{t}\boldsymbol{\omega}$, set T[**A**, **t**] = $\mathbf{t}\boldsymbol{\omega}$.
  - For each production $\mathbf{A} \to \mathbf{B}\boldsymbol{\omega}$, set T[**A**, **t**] = $\mathbf{B}\boldsymbol{\omega}$ for each **t** ∈ FIRST(**B**).

LL(1) with ε-Productions

- What if the grammar has ε-productions?

> Given a nonterminal X, and all its productions $X \rightarrow \gamma_1, X \rightarrow \gamma_2, \ldots, X \rightarrow \gamma_n$ ~~with $\gamma_i \neq \varepsilon$~~, if FIRST($\gamma_1$), …, FIRST($\gamma_n$) all ***mutually disjoint***, then next character tells us which production to use.

- Computation of **FIRST** is different.
  - What if the first nonterminal in a production can produce ε?
- Building the table is different.
  - What action do you take if the correct production produces the empty string?

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | … | 9

Num	Sign	Digit	Digits	More
	+ -	0 5 1 6 2 7 3 8 4 9		

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9		

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | ... | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | … | 9

Nullable(Num) = False
Nullable(Sign) = True
Nullable(Digits) = False
Nullable(More) = True
Nullable(Digit) = False

Num	Sign	Digit	Digits	More
+ -	+ -	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9	0 5 1 6 2 7 3 8 4 9

FIRST Sets for LL(1) with ε-Productions

Num → Sign Digits
Sign → + | − | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | 2 | ... | 9

Nullable(Num) = False
Nullable(Sign) = True
Nullable(Digits) = False
Nullable(More) = True
Nullable(Digit) = False

Num		Sign		Digit		Digits		More	
+	-	+	-	0	5	0	5	0	5
0	5			1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9								

FIRST Sets Computation for LL(1) with ε

- Let $\boldsymbol{\alpha}$ be a string of terminals or non-terminals; $\boldsymbol{\alpha}$ is **nullable** iff $\boldsymbol{\alpha}$ can derive the empty string.

- Initially,
  - for all nonterminals **A**, set $\mathbf{FIRST(A)} = \{\ \mathbf{t} \mid \mathbf{A} \rightarrow \mathbf{t}\boldsymbol{\omega} \text{ for some } \boldsymbol{\omega}\}$
  - For all nonterminals **A** with $\mathbf{A} \rightarrow \boldsymbol{\varepsilon}$, set **Nullable**(**A**) = True.
- Repeat the following until no changes occur:
  - For each $\mathbf{A} \rightarrow \boldsymbol{\alpha}$, if $\boldsymbol{\alpha}$ is nullable, set $\mathbf{Nullable(A)} = \text{True}$.
  - For each $\mathbf{A} \rightarrow \boldsymbol{\alpha}\mathbf{t}\boldsymbol{\omega}$, if $\boldsymbol{\alpha}$ is nullable, set **FIRST**(**A**) = **FIRST**(**A**) ∪ { **t** }
  - For each $\mathbf{A} \rightarrow \boldsymbol{\alpha}\mathbf{B}\boldsymbol{\omega}$, if $\boldsymbol{\alpha}$ is nullable, set **FIRST**(**A**) = **FIRST**(**A**) ∪ (**FIRST**(**B**)).

Build Parsing Tables for LL(1) with ε-Productions

Msg → Hi End
Hi → hello | heya | yo
End → world! | ε

Msg	Hi	End
hello heya yo	hello heya yo	world

FIRST Sets

	hello	heya	yo	world!
Msg				
Hi				
End				

Build Parsing Tables for LL(1) with ε-Productions

```
Msg → Hi End
Hi  → hello | heya | yo
End → world! | ε
```

Msg	Hi	End
hello heya yo	hello heya yo	world

FIRST Sets

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi				
End				

Build Parsing Tables for LL(1) with ε-Productions

Msg → Hi End
Hi → hello | heya | yo
End → world! | ε

Msg	Hi	End
hello heya yo	hello heya yo	world

FIRST Sets

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				

Build Parsing Tables for LL(1) with ε-Productions

Msg → Hi End
Hi → hello | heya | yo
End → world! | ε

Msg	Hi	End
hello heya yo	hello heya yo	world

FIRST Sets

	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

ε is Complicated

Msg $	hello $
Hi End $	hello $
hello End $	hello $
End $	$



	hello	heya	yo	world!
Msg	Hi End	Hi End	Hi End	
Hi	hello	heya	yo	
End				world!

ε is Complicated

- When constructing LL(1) tables with ε-productions, we need to have an extra column for **$**.

Msg → **Hi End**
Hi → **hello** | **heya** | **yo**
End → **world!** | **ε**

Nullable(**Msg**) = False
Nullable(**Hi**) = False
Nullable(**End**) = True

Msg	Hi	End
hello heya yo	hello heya yo	world

FIRST Sets

	hello	heya	yo	world!	$
Msg	Hi End	Hi End	Hi End		
Hi	hello	heya	yo		
End				world!	ε

ε is Complicated

Msg $	**hello** $
Hi End $	**hello** $
hello End $	**hello** $
End $	$



	hello	**heya**	**yo**	**world!**	$
Msg	**Hi End**	**Hi End**	**Hi End**		
Hi	**hello**	**heya**	**yo**		
End				**world!**	**ε**

It Gets Trickier

Num → Sign Digits
Sign → + | - | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | … | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
0 5		1 6	1 6	1 6
1 6		2 7	2 7	2 7
2 7		3 8	3 8	3 8
3 8		4 9	4 9	4 9
4 9				

Nullable(**Num**) = False
Nullable(**Sign**) = **True**
Nullable(**Digits**) = False
Nullable(**More**) = **True**
Nullable(**Digit**) = False

	+	-	#	$
Num	Sign Digits	Sign Digits		
Sign	+	-		
Digits			Digits More	
More			Digits	
Digit			#	

It Gets Trickier

Num	→	Sign Digits
Sign	→	+ | - | ε
Digits	→	Digit More
More	→	Digits | ε
Digit	→	0 | 1 | … | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
0 5		1 6	1 6	1 6
1 6		2 7	2 7	2 7
2 7		3 8	3 8	3 8
3 8		4 9	4 9	4 9
4 9				

Nullable(**Num**) = False
Nullable(**Sign**) = **True**
Nullable(**Digits**) = False
Nullable(**More**) = **True**
Nullable(**Digit**) = False

	+	-	#	$
Num	Sign Digits	Sign Digits	Sign Digits	
Sign	+	-		
Digits			Digits More	
More			Digits	
Digit			#	

It Gets Trickier

Num → Sign Digits
Sign → + | - | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | … | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
0 5		1 6	1 6	1 6
1 6		2 7	2 7	2 7
2 7		3 8	3 8	3 8
3 8		4 9	4 9	4 9
4 9				

Nullable(**Num**) = False
Nullable(**Sign**) = **True**
Nullable(**Digits**) = False
Nullable(**More**) = **True**
Nullable(**Digit**) = False

	+	-	#	$
Num	Sign Digits	Sign Digits	Sign Digits	
Sign	+	-	ε	
Digits			Digits More	
More			Digits	
Digit			#	

It Gets Trickier

Num → Sign Digits
Sign → + | - | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | … | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
0 5		1 6	1 6	1 6
1 6		2 7	2 7	2 7
2 7		3 8	3 8	3 8
3 8		4 9	4 9	4 9
4 9				

Nullable(**Num**) = False
Nullable(**Sign**) = **True**
Nullable(**Digits**) = False
Nullable(**More**) = **True**
Nullable(**Digit**) = False

	+	-	#	$
Num	Sign Digits	Sign Digits	Sign Digits	
Sign	+	-	ε	
Digits			Digits More	
More			Digits	ε
Digit			#	

It Gets Trickier

Num → Sign Digits
Sign → + | - | ε
Digits → Digit More
More → Digits | ε
Digit → 0 | 1 | … | 9

Num	Sign	Digit	Digits	More
+ -	+ -	0 5	0 5	0 5
0 5		1 6	1 6	1 6
1 6		2 7	2 7	2 7
2 7		3 8	3 8	3 8
3 8		4 9	4 9	4 9
4 9				

Nullable(**Num**) = False
Nullable(**Sign**) = **True**
Nullable(**Digits**) = False
Nullable(**More**) = **True**
Nullable(**Digit**) = False

	+	-	#	$
Num	Sign Digits	Sign Digits	Sign Digits	
Sign	+	-	ε	
Digits			Digits More	
More			Digits	ε
Digit			#	

FOLLOW Sets

- With ε-productions, we may have to "look past" the current nonterminal.
- The **FOLLOW set** contains terminals that may immediately follow a nonterminal.
  - **FOLLOW**(**A**) = { **t** | **S** $\Rightarrow^*$ **α**A**t**$\boldsymbol{\omega}$ for some **α**, $\boldsymbol{\omega}$ } where **S** is the start symbol.
- Computation of **FOLLOW** Sets:
  - Initially, for each nonterminal **A**, set **FOLLOW**(**A**) = { **t** | **B** $\rightarrow \alpha$**A**t$\boldsymbol{\omega}$ is a production }
  - Add **$** to **FOLLOW**(**S**), where **S** is the start symbol.
  - Repeat the following until no changes occur:
    - If **B** → α**A**$\boldsymbol{\omega}$ is a production, set **FOLLOW**(**A**) = **FOLLOW**(**A**) ∪ **FIRST***($\boldsymbol{\omega}$).
      - $\boldsymbol{\omega} = \mathbf{t}\gamma \Rightarrow \mathbf{t} \in \mathbf{FIRST}^*(\boldsymbol{\omega})$
      - $\boldsymbol{\omega} = \mathbf{C}\gamma \Rightarrow$ if $\mathbf{Nullable}(\mathbf{C}) =$ True then $\mathbf{FIRST}(\mathbf{C}) \cup \mathbf{FIRST}^*(\boldsymbol{\gamma}) \subseteq \mathbf{FIRST}^*(\boldsymbol{\omega})$
      - $\boldsymbol{\omega} = \mathbf{C}\gamma \Rightarrow$ if $\mathbf{Nullable}(\mathbf{C}) =$ False then $\mathbf{FIRST}(\mathbf{C}) \subseteq \mathbf{FIRST} * (\boldsymbol{\omega})$
    - If **B** → α**A**$\boldsymbol{\omega}$ with **Nullable**($\boldsymbol{\omega}$)=True, set **FOLLOW**(**A**) = **FOLLOW**(**A**) ∪ **FOLLOW**(**B**).

The Final LL(1) Table Algorithm

- Compute **FIRST**(**A**) , and **FOLLOW**(**A**) for all non-terminals **A**.
- For each rule $\mathbf{A} \rightarrow \boldsymbol{\omega}$, and terminal **t** $\in \mathbf{FIRST}(\mathbf{A})$, set T[**A**, **t**] = $\boldsymbol{\omega}$.
  - Note that **ε** is not a terminal.
- For each rule $\mathbf{A} \rightarrow \boldsymbol{\omega}$, if $\boldsymbol{\omega}$ is nullable, set T[**A**, **t**] = $\boldsymbol{\omega}$ for each **t** $\in$ **FOLLOW**(**A**).

A Grammar that is Not LL(1)

- Consider a left-recursive grammar: **A** → **Ab** | **c**
- **FIRST(A)** = {**c**}
- However, we cannot build an LL(1) parse table.
- **Why?**

	b	c
A		A → Ab A → c

- **Cannot uniquely predict production!**
- This is called a **FIRST/FIRST conflict**.

Eliminating Left Recursion

- In general, left recursion can be converted into right recursion by a mechanical transformation.
- Consider the grammar $\mathbf{A} \rightarrow \mathbf{A}\boldsymbol{\omega}|\boldsymbol{\alpha}$
- This will produce $\boldsymbol{\alpha}$ followed by some number of $\boldsymbol{\omega}$'s.
- We can rewrite the grammar as

$$\mathbf{A} \rightarrow \boldsymbol{\alpha}\mathbf{B}$$
$$\mathbf{B} \rightarrow \varepsilon \mid \boldsymbol{\omega}\mathbf{B}$$

Another Non-LL(1) Grammar

- Consider the following grammar:

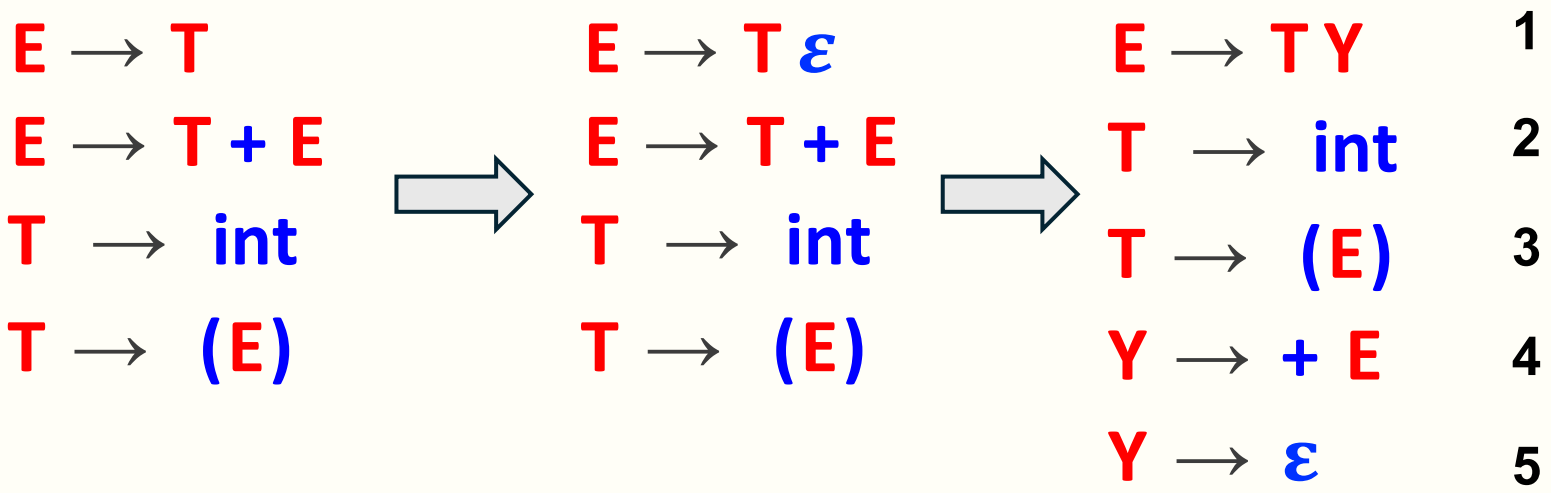
$$E \rightarrow T$$
$$E \rightarrow T + E$$
$$T \rightarrow \text{int}$$
$$T \rightarrow (E)$$



How do you predict which of these to use?

- **FIRST**(**E**) = { **int**, **(** }
- **FIRST**(**T**) = { **int**, **(** }
- Why is this grammar not **LL**(1)?

Left Factoring



Left Factoring

E → T Y	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
	int	+
	(	ε
FOLLOW		
E	T	Y

Left Factoring

E → T Y	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int	int	+
(	(	ε
FOLLOW		
E	T	Y

Left Factoring

E → T Y	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int	int	+
(	(	ε
FOLLOW		
E	T	Y
$	+	$
)		)

Left Factoring

E → T Y	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int	int	+
(	(	ε
FOLLOW		
E	T	Y
$	+	$
)	$	)
	)	

	int	(	)	+	$
E	1	1			
T	2	3			
Y				4	

Left Factoring

E → T Y	1
T → int	2
T → (E)	3
Y → + E	4
Y → ε	5

FIRST		
E	T	Y
int	int	+
(	(	ε
FOLLOW		
E	T	Y
$	+	$
)	$	)
	)	

	int	(	)	+	$
E	1	1			
T	2	3			
Y			5	4	5

A Formal Characterization of LL(1)

- A grammar ***G*** is **LL(1)** iff for any productions $\mathbf{A} \rightarrow \boldsymbol{\omega}_1$ and $\mathbf{A} \rightarrow \boldsymbol{\omega}_2$,
  - the sets **FIRST**($\boldsymbol{\omega}_1$ **FOLLOW**(**A**)) and **FIRST**($\boldsymbol{\omega}_2$ **FOLLOW**(**A**)) are **disjoint**.
- This condition means there are **no conflicts** in the table.
- Algorithm for computing **FIRST**, **FOLLOW**, and **Nullable**
  - **Nullable**(X)= true iff X can derive the empty string
  - **FIRST**(γ)=all terminal symbols that can start a string derived from γ
    - Example: consider FIRST($X\ Y\ Z$)
      - Do we need to know FIRST(X)?
      - if X can derive the empty string, then FIRST(Y) ⊆ FIRST($X\ Y\ Z$)
      - if Y can also derive the empty string, then FIRST(Z) ⊆ FIRST($X\ Y\ Z$)
  - **FOLLOW**(X): all terminals that can immediately follow X
  - Algorithm iterates computing these until a fix point reached
    - Note: knowing nullable(X) and FIRST(X) for all non-terminals X allows us to compute nullable(γ) and FIRST(γ) for arbitrary strings of symbols γ

An LL(1) Parsing Example

❑Example: Computing Nullable

```
S -> E eof
E -> T E'
E' -> + T E'
E' -> - T E'
E' -> ε
T -> F T'
T' -> * F T'
T' -> / F T'
T' -> ε
F -> id
F -> num
F -> (E)
```

	nullable	FIRST	FOLLOW
S	$\bot$		
E	$\bot$		
E'	$\top$		
T	$\bot$		
T'	$\top$		
F	$\bot$		

X is nullable if there is a production $X \rightarrow \gamma$ where γ is empty, or γ is all nullable nonterminals.

T' and E' are nullable!

And, we've finished nullable. Why?

An LL(1) Parsing Example

- Example: Computing FIRST

```
S -> E eof
E -> T E'
E' -> + T E'
E' -> - T E'
E' -> ε
T -> F T'
T' -> * F T'
T' -> / F T'
T' -> ε
F -> id
F -> num
F -> (E)
```

Given production $X \to t\gamma, t \in \mathbf{FIRST}(X)$

	nullable	FIRST	FOLLOW
S	$\bot$		
E	$\bot$		
E'	$\top$	+ -	
T	$\bot$		
T'	$\top$	* /	
F	$\bot$	id num (	

Given production $X \to \gamma Y \sigma$, if $Nullable(\gamma)$ then $FIRST(Y) \subseteq FIRST(X)$.

	nullable	FIRST	FOLLOW
S	$\bot$	id num (	
E	$\bot$	id num (	
E'	$\top$	+ -	
T	$\bot$	id num (	
T'	$\top$	* /	
F	$\bot$	id num (	

An LL(1) Parsing Example

❑Example: Computing FOLLOW

```
S -> E eof
E -> T E'
E' -> + T E'
E' -> - T E'
E' -> ε
T -> F T'
T' -> * F T'
T' -> / F T'
T' -> ε
F -> id
F -> num
F -> (E)
```

	nullable	FIRST	FOLLOW
S	$\perp$	id num (	
E	$\perp$	id num (	eof)
E'	$\top$	+ -	eof)
T	$\perp$	id num (	+ - eof)
T'	$\top$	* /	+ - eof)
F	$\perp$	id num (	* / + - eof)

Given production $X \rightarrow \gamma Z \delta \sigma$,

- $FIRST(\delta) \subseteq FOLLOW(Z)$
- and if δ is nullable then $FIRST(\sigma) \subseteq FOLLOW(Z)$
- and if $\delta\sigma$ is nullable then $FOLLOW(X) \subseteq FOLLOW(Z)$

Repeat until no more changes …

Constructing the Predictive Parsing Table

For $X \rightarrow \gamma$, add $X \rightarrow \gamma$ to row X column t for all $t \in FIRST(\gamma)$;
For $X \rightarrow \gamma$, if γ is nullable, add $X \rightarrow \gamma$ to row X column t for all $t \in FOLLOW(X)$.

```
S -> E eof
E -> T E'
E' -> + T E'
E' -> - T E'
E' -> ε
T -> F T'
T' -> * F T'
T' -> / F T'
T' -> ε
F -> id
F -> num
F -> (E)
```

	nullable	FIRST	FOLLOW
S	$\perp$	id num (	
E	$\perp$	id num (	eof)
E'	$\top$	+ -	eof)
T	$\perp$	id num (	+ - eof)
T'	$\top$	* /	+ - eof)
F	$\perp$	id num (	* / + - eof)

	id	num	+	-	*	/	(	)	eof
S	$S \rightarrow E\ eof$	$S \rightarrow E\ eof$					$S \rightarrow E\ eof$		
E	$E \rightarrow T\ E'$	$E \rightarrow T\ E'$					$E \rightarrow T\ E'$		
E'			$E' \rightarrow +\ T\ E'$	$E' \rightarrow -\ T\ E'$				$E' \rightarrow$	$E' \rightarrow$
T	$T \rightarrow F\ T'$	$T \rightarrow F\ T'$					$T \rightarrow F\ T'$		
T'			$T' \rightarrow$	$T' \rightarrow$	$T' \rightarrow *\ F\ T'$	$T' \rightarrow /\ F\ T'$		$T' \rightarrow$	$T' \rightarrow$
F	$F \rightarrow id$	$F \rightarrow num$					$F \rightarrow (E)$		

Predictive Parsing Table

- If each cell contains at most one production,
- parsing is predictive! Table tells us exactly which production to apply.
- Go back and see how to use this table.

The Strengths of LL(1)

- LL(1) is Straightforward
  - Can be implemented quickly with a table- driven design.
  - Can be implemented by recursive descent:
    - Define a function for each nonterminal.
    - Have these functions call each other based on the lookahead token.

- LL(1) is Fast
  - Both table-driven LL(1) and recursive-descent-powered LL(1) are fast.
  - Can parse in $O(n|G|)$ time, where n is the length of the string and $|G|$ is the size of the grammar.

LL(1), LL(k), LL(*)

- Grammars whose predictive parsing table contains at most one production per cell are called $\mathbf{LL(1)}$
  - First **L**: Left-to-right parsing, i.e., go through token stream from left to right.
  - Second **L**: Leftmost derivation, derivation expands the leftmost non-terminal
  - **1**: 1-symbol lookahead
- Can be generalized to $\mathbf{LL(2)}$, $\mathbf{LL(3)}$, etc.
  - Columns of the predictive parsing table have $\boldsymbol{k}$ tokens
  - $\mathbf{FIRST}(\mathbf{X})$ generalized to $\mathbf{FIRST}$-$k(\mathbf{X})$
- An $\mathbf{LL}(*)$ grammar can determine next production using finite (but maybe unbounded) lookahead
- An ambiguous grammar is not $\mathbf{LL}(\boldsymbol{k})$ for any k, or even $\mathbf{LL}(*)$
  - **Why?**

Summary of Top-down Parsing

- **Top-down parsing** derives the input from the start symbol.
- **Leftmost BFS** is mainly of theoretical interest.
- **Leftmost DFS** is uncommon in practice.
- **LL(k)** reads left to right and uses k token of lookahead to construct a leftmost derivation.
- **FIRST sets** contain terminals that may be the first symbol of a production.
- **FOLLOW sets** identifies the terminals that can immediately follow a nonterminal in a sentential form.
- Eliminating **left recursion** or applying **left factoring** can help make a grammar suitable for $\mathbf{LL}(k)$ parsing.

References

- Stanford University, CS143: Compilers — Top-Down Parsing, Lecture Slides, 2012. https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/03/Slides03.pdf
- Harvard University, CS153: Compilers — LL Parsing, Lecture Slides, 2019. https://groups.seas.harvard.edu/courses/cs153/2019fa/lectures/Lec10-LL-Parsing.pdf

练习： LL(1) Parsing

❑利用构造好的解析表解析: **(id+num) eof**

```
S -> E eof
E -> T E'
E' -> + T E'
E' -> - T E'
E' -> ε
T -> F T'
T' -> * F T'
T' -> / F T'
T' -> ε
F -> id
F -> num
F -> (E)
```

	id	num	+	-	*	/	(	)	eof
S	S → E eof	S → E eof					S → E eof		
E	E → T E'	E → T E'					E → T E'		
E'			E' → + T E'	E' → - T E'				E' →	E' →
T	T → F T'	T → F T'					T → F T'		
T'			T' →	T' →	T' → * F T'	T' → / F T'		T' →	T' →
F	F → id	F → num					F → (E)		

- 文法开始符是**S**
- 要求写清楚每一步的解析过程